Arduino Project: 3 Scale Theremin

Everton Smith

everton wsmith.github.io/Everton/projects

https://www.tinkercad.com/things/2bUNRqS9FN4-trivia?sharecode=mQm1TOrS0dfiZms-TswxxG1CJVHEV9KaQx-uh86eyvs

Table of Contents

I. ABSTRACT 2

II. METHODOLOGY: KEY-SWITCHING LOGIC 2

III. CALIBRATED FREQUENCY DATA (LUT) 2

IV. DIAGRAMS 4

V. CONCLUSION 7

VI. COMMENTS 7

VII. FINAL VERIFIED CODE 8

Table of Contents

Figure 1: Schematic Diagram 4

Figure 2: Wiring Diagram 5

Figure 3: Hardware - C scale mode 5

Figure 4: Hardware - G scale mode 6

Figure 5: Hardware - F scale mode 6

I. ABSTRACT

This project demonstrates the design of a digital Theremin capable of real-time switching between C Major, F Major, and G Major scales. Utilizing an HC-SR04 ultrasonic sensor for distance-to-pitch mapping, the system employs a recursive digital low-pass filter to stabilize output. By implementing dedicated Look-Up Tables (LUT) for different musical keys, the device showcases the flexibility of software-defined instrumentation in embedded systems.

II. METHODOLOGY: KEY-SWITCHING LOGIC

- Scale Selection: Instead of a single frequency array modified by a multiplier, the system holds three independent float arrays.
- Memory Efficiency: While using multiple arrays increases Flash memory usage, it allows for musical "Sharps" and "Flats" (like Bb in F Major or F# in G Major) that simple multiplication cannot achieve.
- State Machine: The three tactile buttons function as a simple state machine, assigning the activeScale pointer (see code) to the corresponding memory address of the chosen key.

III. CALIBRATED FREQUENCY DATA (LUT)

The following table represents the frequencies programmed into the microcontroller's memory for each selectable key.

Note Index	C Major (Hz)	F Major (Hz)	G Major (Hz)
0	261.63	349.23	392.00
1	293.66	392.00	440.00
2	329.63	440.00	493.88
3	349.23	466.16	523.25
4	392.00	523.25	587.33
5	440.00	587.33	659.25
6	493.88	659.25	739.99
7	523.25	698.46	783.99

IV. Diagrams

Figure 1: Schematic Diagram

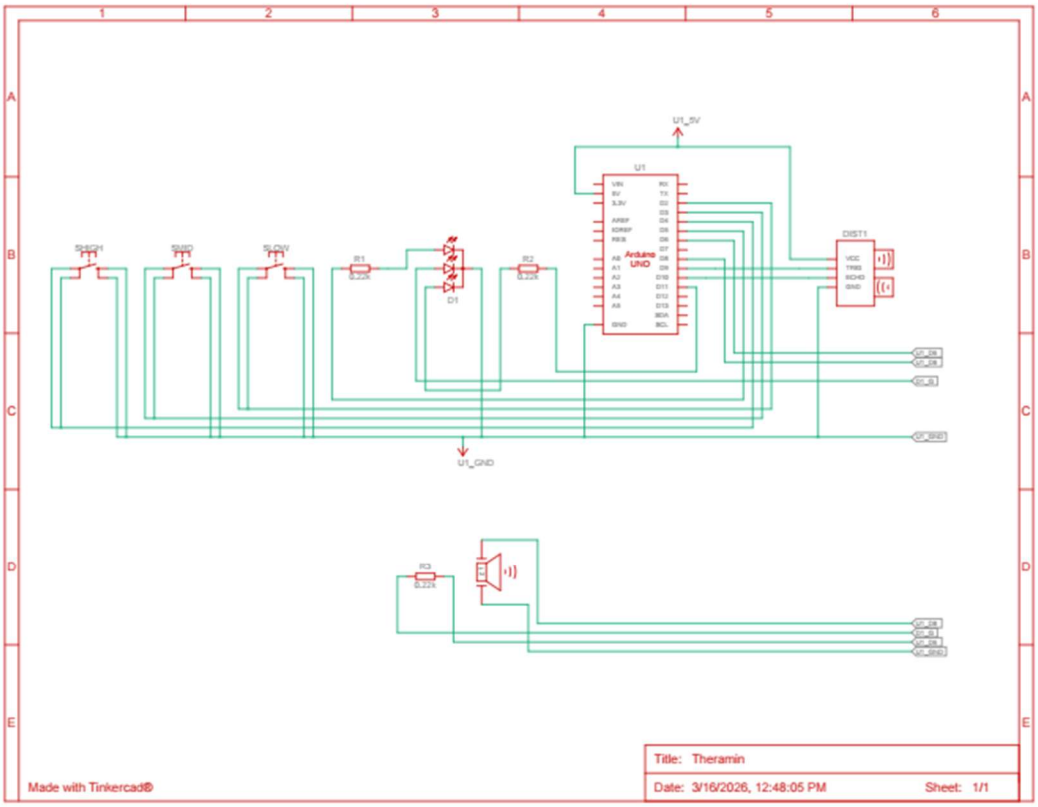

Figure 2: Wiring Diagram

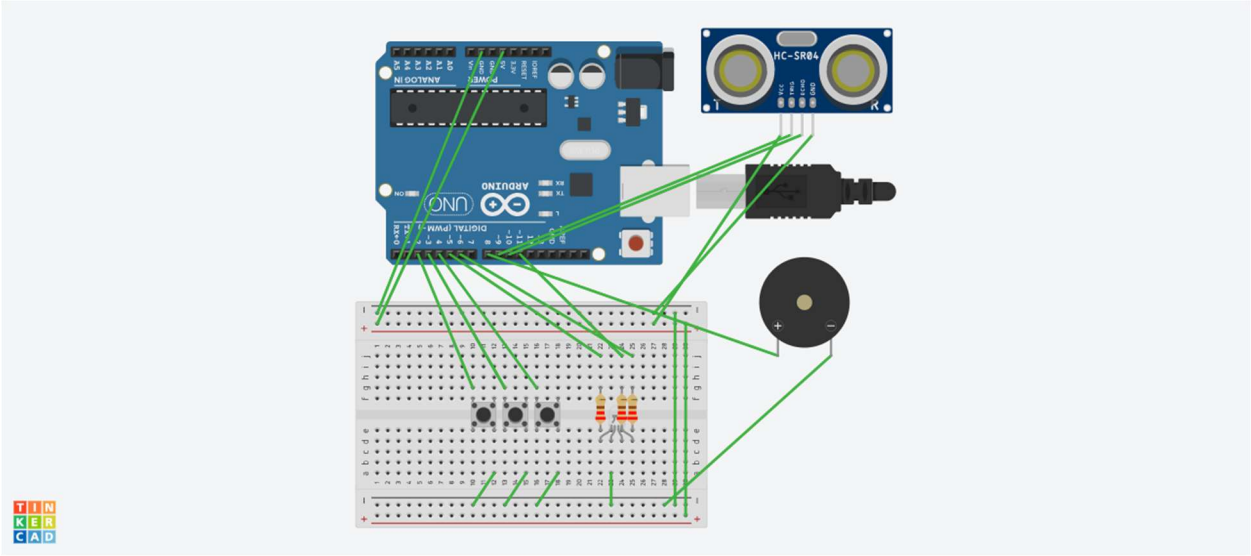

Figure 3: Hardware - C scale mode

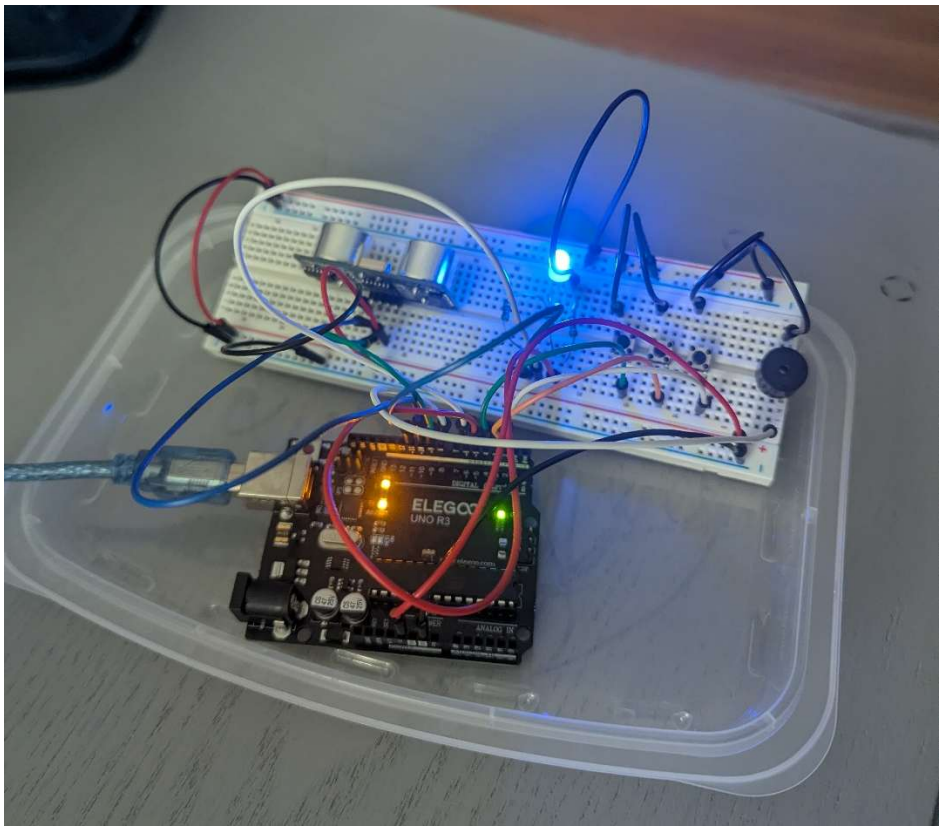

Figure 4: Hardware - G scale mode

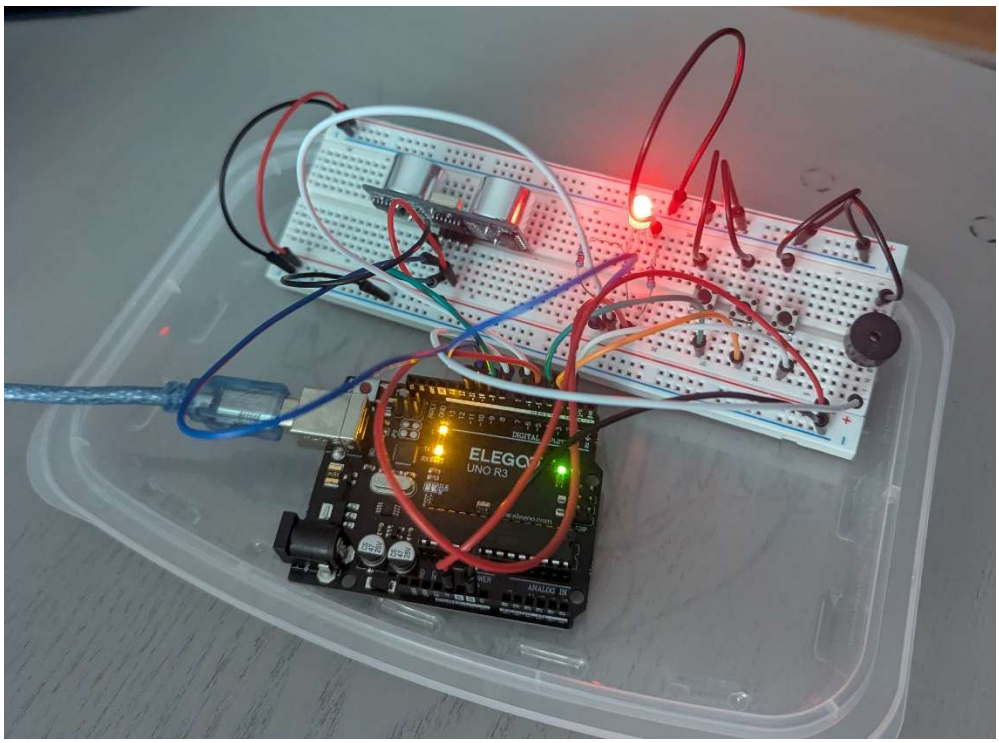

Figure 5: Hardware - F scale mode

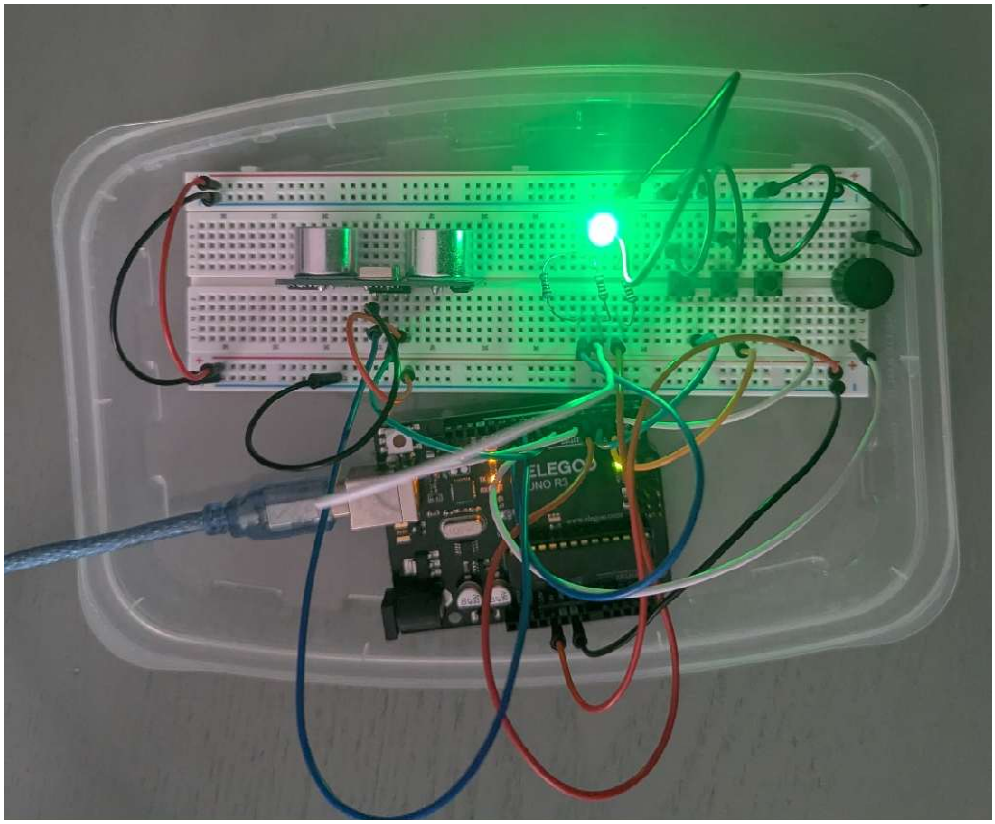

Table 1: Components List

Name	Quantity	Component
U1	1	Arduino Uno R3
PIEZO1	1	Piezo
Slow, Smid, Shigh	3	Pushbutton
D1	1	RCBG LED RGB
R1, R2, R3	3	0.22 kΩ Resistor
DIST1	1	Ultrasonic Distance Sensor (4-pin)

V. CONCLUSION

By transitioning from octave scaling to key-specific arrays, the device has improved musical versatility and stability. Adding the Low-Pass Filter was critical in ensuring that dead zones between 3-inch intervals were eliminated, providing a stable response.

VI. Comments

During the testing phase, a peripheral conflict was observed between the tone() library and the PWM output on Pin 11. Since both functions utilize Timer 2 on the ATmega328P microcontroller, resource contention occurred during the playback of the lowest frequency (261.63 Hz). This resulted in erratic visual feedback on the Blue LED channel. The issue was resolved by migrating the Blue LED to a Timer 0 PWM pin (Pin 5), successfully decoupling the audio generation from the visual feedback system.

VII. FINAL VERIFIED CODE

```
/**
 * Project: Stabilized Three-Octave Theremin
 * Features: Moving Average Filter for sensor stability, Hardcoded C/F/G Scales.
 */

// Assign sensor and buzzer pins
const int trigPin = 9;
const int echoPin = 10;
const int buzzerPin = 8;

// RGB Pins (PWM)
const int redPin = 5;
const int greenPin = 6;
const int bluePin = 11;

// Buttons
const int btnLow = 2;
const int btnMid = 3;
const int btnHigh = 4;

// Specific Scale Arrays (C Maj, F Maj, G Maj)
float C_scale[] = { 261.00, 293.66, 329.63, 349.23, 392.00, 440.00, 493.88, 523.25 };
float F_scale[] = { 349.23, 392.00, 440.00, 466.16, 523.25, 587.33, 659.25, 698.46 };
float G_scale[] = { 392.00, 440.00, 493.88, 523.25, 587.33, 659.25, 739.99, 783.99 };

// Filtering Variables
```

```
float smoothedDist = 24.0;
const float filterWeight = 0.15;  // Lower = smoother but slower response. Higher = snappier but
noisier.
int currentOctave = 1;          // Default to

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(btnLow, INPUT_PULLUP);
  pinMode(btnMid, INPUT_PULLUP);
  pinMode(btnHigh, INPUT_PULLUP);
  Serial.println("System Ready.");
}

void loop() {
  // Octave Selection
  if (digitalRead(btnLow) == LOW) {
    currentOctave = 0;
    Serial.println("C Scale Active");
  }
  if (digitalRead(btnMid) == LOW) {
    currentOctave = 1;
    Serial.println("F Scale Active");
  }
  if (digitalRead(btnHigh) == LOW) {
    currentOctave = 2;
```

```
  Serial.println("G Scale Active");
}

// Distance Measurement
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

long duration = pulseIn(echoPin, HIGH);
float rawDist = (float)duration / 74.0 / 2.0;

// Apply Low-Pass Filter (Moving Average)
// This prevents "random" notes caused by sensor jitter
// This took trial and error, other methods may be better
if (rawDist > 0 && rawDist < 100) {  // Filter out unrealistic sensor errors
  smoothedDist = (rawDist * filterWeight) + (smoothedDist * (1.0 - filterWeight));
}

// Playback Logic
// Use a slightly wider "Active Zone" (0 to 26) to prevent the edge-flicker
if (smoothedDist > 0.1 && smoothedDist < 26.0) {

  // Narrow down range
  int noteIndex;
  if (smoothedDist > 21.000) noteIndex = 0;  // The "Far" note (C4 in Blue Mode)
  else if (smoothedDist > 18.000) noteIndex = 1;
  else if (smoothedDist > 15.000) noteIndex = 2;
  else if (smoothedDist > 12.000) noteIndex = 3;
  else if (smoothedDist > 9.000) noteIndex = 4;
```

```
else if (smoothedDist > 6.000) noteIndex = 5;
else if (smoothedDist > 3.000) noteIndex = 6;
else noteIndex = 7;

// Frequency Selection
float frequency;
switch (currentOctave) {
  case 0: frequency = C_scale[noteIndex]; break;
  case 1: frequency = F_scale[noteIndex]; break;
  case 2: frequency = G_scale[noteIndex]; break;
  default: frequency = C_scale[noteIndex]; break;
}

// Sound Output
tone(buzzerPin, (int)frequency);

// More consistent lighting
int brightness;
if (smoothedDist > 18.0) brightness = 180;
else brightness = 255;

updateLED(currentOctave, brightness);

// // Debugging - This will tell us EXACTLY what the Arduino sees
// Serial.print(F("Dist: "));
// Serial.print(smoothedDist);
// Serial.print(F(" | Octave: "));
// Serial.print(currentOctave);
// Serial.print(F(" | Index: "));
// Serial.println(noteIndex);
```

```
  } else {
    noTone(buzzerPin);
    updateLED(-1, 0);
  }

  delay(30);  // Faster loop for better response with the filter
}

void updateLED(int octave, int bright) {
  analogWrite(redPin, 0);
  analogWrite(greenPin, 0);
  analogWrite(bluePin, 0);

  if (octave == 0) analogWrite(bluePin, bright);   // C = Blue
  if (octave == 1) analogWrite(greenPin, bright);  // F = Green
  if (octave == 2) analogWrite(redPin, bright);    // G = Red
}
```